CONFIDENTIAL — DEVELOPER HANDOFF

DocCompare

Technical Handoff Document
FastAPI + MongoDB Migration & Product Roadmap

Version 1.0 · June 2026

Repository: doccompare (React + Hono/tRPC/MySQL → target: React + FastAPI/MongoDB)

Prepared for: Client development team

Table of Contents

1. Executive Summary

DocCompare is an AI-powered document audit platform for the toy and children's product import industry. It validates packaging documentation across three sequential departments:

```
Atendimento (Phase 1)  →  Design (Phase 2)  →  CQ / Quality Control (Phase 3)  →  Concluído
```

Aspect	Status
Overall completion	~60–70% — core comparison workflow works; phase-specific UX incomplete
Phase 0 (Foundation)	Done — auth, DB, upload, AI, workflow, deploy
Phase 1 (Atendimento / IA Validator)	~85% Done — polish items remain
Phase 2 (Design Review)	Not Built — primary next product phase
Phase 3 (CQ Review)	Prompt Only — no dedicated UX
Platform migration agreed	**FastAPI (Python) + MongoDB** — replace Hono/tRPC/MySQL
Frontend	**Keep React 19 + Vite** — rewire API client from tRPC to REST/OpenAPI
AI model (non-negotiable)	**Kimi k2.5** via Moonshot API (`https://api.moonshot.ai`)

> **Developer mandate:** Port the existing business logic to FastAPI with MongoDB, preserve JSON response shapes where possible to minimize frontend churn, then implement Phase 2 Design features per RFQ.

2. Product Overview & Business Workflow

2.1 What the application does

1. User uploads documents (Order Details, Die Cut, photos, PO, artwork, etc.)
2. System extracts text via OCR pipeline (pdf-parse → Mistral OCR 3 → Tesseract.js)
3. Phase 1 runs programmatic validation (CNPJ, EAN-13, DUN-14, NCM) before AI
4. Kimi k2.5 compares documents and returns structured JSON per packaging type
5. Human reviewer approves/rejects each packaging (barcode_label, color_box, master_carton)
6. When all three approved → order advances to next department phase
7. 20% of approved items randomly assigned for AQL human QA review

2.2 Three packaging types (always evaluated together)

- `barcode_label` — Barcode label
- `color_box` — Color box / inner packaging
- `master_carton` — Master carton / outer box

2.3 Document types (upload classification)

tipoDocumento	Description	Phase
order_details	Order Details (Excel) — reference base	All
die_cut	Structural die cut PDF	1
foto	Product photo	1, 3
commercial_invoice	Purchase Order (PO)	1
relatorio_inspecao	Prior inspection report (optional)	1
artwork	Developed artwork PDF	2
contraprova	Supplier sketch / proof	3
briefing, packing_list, bill_of_lading, outro	Other / misc	—

2.4 AI Prompts (3 active — seed via setup endpoint)

Slug	Department	Purpose
briefing-validacao-inicial	atendimento	OD × PO × Die Cut × Photo — Phase 1 briefing
revisao-embalagens	design	Artwork vs Order Details
revisao-sketch	cq	Supplier sketch vs approved artwork

2.5 Critical business rules

- **PO > Order Details** — Purchase Order reflects client intent; takes priority on conflict
- **Inspection report** — optional except for re-shipments/repeats
- **Validation does not auto-block** — flags issues in PDF/report; human approves
- **QR Code rule** — height > width → lateral face; width > height → top face
- **Phase 1 temperature** — 0.3 (deterministic AI)

3. Current Architecture & Code Structure

3.1 Stack today vs. target

Layer	Current	Target
Frontend	React 19, Vite, Tailwind, shadcn/ui, tRPC client	React 19 (keep) — REST + React Query / OpenAPI client
Backend	Hono, tRPC, Drizzle ORM, Node.js	**FastAPI**, Pydantic, Motor/PyMongo
Database	MySQL 8.0 (Docker on VPS)	**MongoDB** (Atlas or self-hosted)
AI	Kimi k2.5 REST	Same — httpx to api.moonshot.ai
OCR	pdf-parse, Mistral OCR, Tesseract.js	pypdf/pdfplumber, Mistral SDK, pytesseract
PDF reports	Playwright (HTML → PDF)	Playwright-Python or WeasyPrint
Deploy	Ubuntu VPS, PM2, GitHub Actions	Uvicorn/Gunicorn + PM2 or systemd

3.2 Repository folder structure

```
doccompare/
├── src/                    # React frontend
│   ├── App.tsx             # Route definitions
│   ├── pages/              # Dashboard, NovaComparacao, Resultado, Historico, Admin...
│   ├── components/layout/  # Sidebar, AppLayout, NotificationBell
│   ├── hooks/useAuth.ts    # JWT session
│   └── providers/trpc.tsx  # API client (REPLACE with REST)
├── api/                    # Node backend (TO BE REPLACED)
│   ├── boot.ts             # Hono server
│   ├── router.ts           # 14 tRPC routers
│   ├── middleware.ts       # Auth middleware
│   ├── comparacao-router.ts # ⭐ Core AI comparison
│   ├── upload-router.ts    # Upload + OCR
│   ├── workflow-router.ts  # Phase transitions
│   ├── lib/                # Validators, OCR, AQL service
│   └── templates/          # PDF HTML templates
├── db/schema.ts            # MySQL schema (reference for MongoDB design)
├── RFQ_DOC_COMPARE.md      # Client requirements Phase 2–5
└── ESPECIFICACAO_FASE1_FINAL.md
```

3.3 Request flow today

```
Browser (React)
    → tRPC HTTP POST /api/trpc/{router}.{procedure}
    → Hono (boot.ts)
    → middleware (JWT verify)
    → router handler
    → Drizzle ORM → MySQL
    → (comparacao) Kimi API → save JSON results
```

3.4 Target request flow

```
Browser (React)
    → REST /api/v1/...  (OpenAPI documented)
    → FastAPI + JWT dependency
    → service layer (ocr, validators, kimi, pdf)
    → Motor/PyMongo → MongoDB
    → GridFS or S3 for file binaries
```

4. User Roles & Permissions Matrix

4.1 Three access-control dimensions

Users are NOT defined by a single role field. The system uses three fields:

Field	Values	Purpose
`role`	`user`, `admin`	System administrator flag
`departamento`	atendimento, design, cq, supervisor, admin	Workflow department assignment
`isSupervisor`	true / false	Elevated cross-phase permissions

4.2 Effective user types

Type	Config	Capabilities
Regular user	role=user, dept set, isSupervisor=false	Standard workflow; advance phase only if dept matches faseAtual
Supervisor	isSupervisor=true	Override reproved items; advance any phase; reprove phase; AQL pool
Admin	role=admin	All admin pages, user/prompt CRUD, delete pedidos, full metrics
Admin + Supervisor	Both	Documented default: admin@doccompare.com / admin123

4.3 Permission matrix by feature

Feature	User	Supervisor	Admin
Login / Dashboard / Nova Comparação	✅	✅	✅
Run any AI prompt (any phase)	✅ ⚠️	✅	✅
View resultado / approve packaging	✅	✅	✅
Override approve reproved item	❌	✅	✅
Avançar fase (workflow)	✅ if dept matches	✅	✅
Reprovar fase	❌	✅	✅
Own notifications / AQL reviews	✅	✅	✅
Admin prompts / users / divergencias	❌	❌	✅
Delete pedido	❌	❌	✅
Advanced dashboard metrics	❌	❌	✅

⚠️ **Permission gaps to fix in FastAPI rewrite:**

- `pedido.list` returns ALL orders — not filtered by userId
- `comparacao.getByUuid` has no ownership check — UUID is sufficient to read
- Department middleware (`atendimentoQuery` , etc.) is defined but **never used** on routes
- Admin pages hidden in sidebar only — no frontend route guard

5. Frontend Pages & Features

Page	Route	Access	Description
Login	/login	Public	Auth, register, forgot password UI
Dashboard	/dashboard	Authed	KPIs, charts (admin), recent pedidos
Nova Comparação	/nova-comparacao	Authed	3-step wizard: upload → prompt → execute
Resultado	/resultado/:uuid	Authed	AI results, approve, PDF, workflow
Historico	/historico	Authed	Order list, search, delete (admin)
Notificações	/notificacoes	Authed	Notification inbox
Revisões AQL	/revisoes-aql	Authed	20% QA review queue
Admin Prompts	/admin/prompts	Admin	CRUD AI prompts
Admin Usuários	/admin/usuarios	Admin	User management
Divergências	/admin/divergencias	Admin API	IA vs human tracking

5.1 Nova Comparação wizard steps

1. **Step 1 — Documents:** Drag-drop files; auto-suggest `tipoDocumento`; user confirms via dropdown
2. **Step 2 — Analysis type:** Select one of 3 prompts from DB
3. **Step 3 — Execute:** Enter pedido code/name → creates pedido → uploads → runs Kimi

6. Backend API Inventory — tRPC → FastAPI Mapping

Current base path: `POST/GET /api/trpc/{router}.{procedure}`
Target base path: `/api/v1/...` (REST, OpenAPI at `/docs`)

6.1 Auth (`auth` → `/api/v1/auth`)

Current tRPC	FastAPI (proposed)	Auth
auth.register	POST /auth/register	Public
auth.login	POST /auth/login	Public → JWT
auth.me	GET /auth/me	Bearer JWT
auth.logout	POST /auth/logout	Client-side token clear
auth.forgotPassword	POST /auth/forgot-password	Public (email not implemented)
auth.resetPassword	POST /auth/reset-password	Public + token

6.2 Pedidos (`pedido` → `/api/v1/pedidos`)

Current tRPC	FastAPI	Auth
pedido.create	POST /pedidos	JWT
pedido.list	GET /pedidos	JWT — FIX: filter by user/dept
pedido.listAll	GET /pedidos/all	Admin
pedido.getById	GET /pedidos/{id}	JWT
pedido.updateStatus	PATCH /pedidos/{id}	Owner or admin
pedido.arquivar	POST /pedidos/{id}/arquivar	Owner or admin
pedido.duplicar	POST /pedidos/{id}/duplicar	JWT
pedido.delete	DELETE /pedidos/{id}	Admin

6.3 Comparações (`comparacao` → `/api/v1/comparacoes`) ⭐ CORE

Current tRPC	FastAPI	Notes
comparacao.create	POST /comparacoes	Links to pedidoId + promptId
comparacao.executar	POST /comparacoes/{uuid}/executar	OCR + validators + Kimi — longest operation
comparacao.getByUuid	GET /comparacoes/{uuid}	Returns pedido, docs, itens, analises
comparacao.checkStatus	GET /comparacoes/{uuid}/status	Polling while processando
comparacao.aprovarItem	POST /comparacoes/itens/{id}/aprovar	status: aprovado|reprovado|parcial
comparacao.aprovarAnaliseItem	POST /comparacoes/analises/{id}/aprovar	Per-field approval (UI not wired)
comparacao.listByPedido	GET /pedidos/{id}/comparacoes	Version history
comparacao.listAll	GET /comparacoes	Admin
comparacao.delete	DELETE /comparacoes/{id}	Owner or admin

6.4 Upload & Documents

Current	FastAPI
upload.register	POST /pedidos/{id}/documentos (multipart or base64)
documento.*	GET/PATCH/DELETE /pedidos/{id}/documentos/{docId}

6.5 Workflow, PDF, Notifications, AQL, Admin

Router	Key endpoints → FastAPI
workflow	GET /pedidos/{id}/workflow · POST /pedidos/{id}/avancar-fase · POST /pedidos/{id}/reprovar-fase
pdf	POST /comparacoes/itens/{id}/pdf · POST /pedidos/{id}/resumo-executivo-pdf
notificacao	GET /notificacoes · PATCH /notificacoes/{id}/lida
aql	GET /aql/minhas-revisoes · POST /aql/revisoes/{id}/submeter
prompt	GET /prompts · Admin CRUD /admin/prompts
usuario	Admin CRUD /admin/usuarios
divergencia	POST /divergencias · GET /admin/divergencias
metricas	GET /metricas/overview · GET /metricas/* (admin charts)
setup	POST /setup/seed-prompts (migration/bootstrap)

6.6 Kimi API integration (preserve exactly)

```
POST https://api.moonshot.ai/v1/chat/completions
Authorization: Bearer {APP_ID}   # sk-... Moonshot API key
Body: {
  "model": "kimi-k2.5",
  "temperature": 0.3,
  "response_format": { "type": "json_object" },
  "messages": [
    { "role": "system", "content": "..." },
    { "role": "user", "content": "..." }
  ]
}
```

7. Database — MySQL Today → MongoDB Target

7.1 MySQL tables today (10 entities)

users · pedidos · prompts · comparacoes · comparacao_itens · analise_itens · documentos · divergencias · revisoes_aql · notificacoes

7.2 MongoDB design principle

> **Do NOT mirror 10 SQL tables.** Design around access patterns. Core rule: *data accessed together should be stored together.*

7.3 Recommended collections

Collection: `users`

```
{
  "_id": ObjectId,
  "email": "user@company.com",     // unique index
  "name": "...",
  "passwordHash": "bcrypt...",
  "role": "user|admin",
  "departamento": "atendimento|design|cq|supervisor|admin",
  "isSupervisor": false,
  "createdAt": ISODate,
  "lastSignInAt": ISODate
}
```

Collection: `prompts`

```
{
  "_id": ObjectId,
  "slug": "briefing-validacao-inicial",  // unique index
  "nome": "...",
  "departamento": "atendimento",
  "promptSistema": "...",
  "modeloOutput": "...",
  "variaveis": [...],
  "ativo": true,
  "versao": 1
}
```

Collection: `pedidos` (MAIN AGGREGATE)

```
{
  "_id": ObjectId,
  "codigoPedido": "SAT11675-25",       // unique index
  "nome": "...",
  "statusGeral": "em_andamento",
  "faseAtual": "atendimento",
  "createdBy": ObjectId("user"),
  "dadosCliente": { "fornecedor": "...", "numeroPO": "..." },

  "documentos": [{
    "id": "uuid",
    "tipoDocumento": "order_details",
    "nomeOriginal": "OD.xlsx",
    "mimeType": "...",
    "gridfsId": ObjectId,              // file binary — NOT inline
    "conteudoExtraidoRef": ObjectId,   // if text large
    "ordem": 0,
    "uploadedAt": ISODate
  }],

  "comparacoes": [{
    "uuid": "...",
    "versao": 1,
    "departamento": "atendimento",
    "promptSlug": "briefing-validacao-inicial",
    "status": "concluido",
    "modelo": "kimi-k2.5",
    "tokensEntrada": 12000,
    "tempoProcessamento": 45,
    "itens": [{
      "tipoEmbalagem": "barcode_label",
      "status": "pendente",
      "resumoExecutivo": "markdown...",
      "secoes": [{ "titulo": "...", "severidade": "critical", "conteudo": "..." }],
      "analises": [{
        "campo": "EAN-13",
        "valorEsperado": "...",
        "valorEncontrado": "...",
        "status": "critical",
        "checklistStatus": "pending"   // NEW Phase 2: ok|nok|pending
      }],
      "divergencias": [...],
      "revisaoAql": { "designadoPara": ObjectId, "status": "pendente" }
    }]
  }],

  "createdAt": ISODate,
  "updatedAt": ISODate
}
```

Collection: `notifications` (separate — unbounded)

```
{ "userId": ObjectId, "tipo": "proxima_fase", "titulo": "...", "lida": false, "createdAt": ISODate }
```

Collection: `document_contents` (optional — large OCR text)

Store OCR text here if >1MB to stay under MongoDB 16MB document limit.

7.4 Required indexes

Collection	Index
users	`{ email: 1 }` unique
prompts	`{ slug: 1 }` unique
pedidos	`{ codigoPedido: 1 }` unique, `{ faseAtual: 1 }`, `{ "comparacoes.uuid": 1 }`
notifications	`{ userId: 1, lida: 1, createdAt: -1 }`

8. What Is Done vs. Not Done

8.1 Completed ✅

Area	Status
React UI (11 pages, shadcn/ui)	✅ Done
JWT auth (local email/password)	✅ Done
MySQL schema + Drizzle ORM	✅ Done (to be replaced)
14 tRPC routers	✅ Done (to be replaced)
Upload + OCR pipeline	✅ Done
Programmatic validators (CNPJ, EAN, DUN, NCM)	✅ Done
Kimi k2.5 comparison + JSON parse	✅ Done
3 packaging results per comparison	✅ Done
Approve/reprove + supervisor override	✅ Done
Phase workflow + notifications	✅ Done
AQL 20% sampling	✅ Done
PDF per packaging item	✅ Done
Admin: prompts, users, divergencias	✅ Done
Document type classification on upload	✅ Done
3 new AI prompts (setup.seed)	✅ Done
GitHub Actions deploy + PM2	✅ Done

8.2 Phase 1 gaps ⚠️

- Resumo Executivo PDF — backend exists, not wired in UI
- True multimodal AI — only OCR text sent to Kimi, not images
- Required document type validation not enforced
- File binaries not stored (text only in DB)
- Password reset email not implemented
- No automated tests

8.3 Phase 2 — Design (NOT BUILT — primary next phase)

- Visual OCR optimized for artwork PDFs
- Interactive checklist: OK / NOK / Pending per field with reviewer + timestamp
- Multiple artwork version upload + history UI

- Phase 1 Executive Summary visible in Design
- Mark Phase 1 items as "Addressed" or "Ignored with reason"

8.4 Phase 3 — CQ (NOT BUILT)

- Dedicated sketch vs artwork flow
- Physical product photo validation (all angles)
- Approved / Rejected / **Conditional** final status

8.5 Phase 4 — Metrics (NOT BUILT)

- Time per phase KPIs · Rework rate · Supplier quality score · Monthly auto-report PDF

9. Known Bugs & Technical Debt

Issue	Severity	Fix in FastAPI rewrite
pedido.list returns all users' orders	High	Filter by userId or department
comparacao.getByUuid no access control	High	Verify ownership or role
Any user can run any phase prompt	Medium	Validate prompt.departamento === pedido.faseAtual
Department middleware unused	Medium	Implement FastAPI dependencies
Admin routes not guarded on frontend	Medium	Route guards + API checks
db/seed.ts has old 5 prompts	Low	Use setup.seed or MongoDB seed script
divergencia.create has no UI on Resultado	Low	Add report button in Phase 2
New pedido per comparison (no continue)	Medium	Support comparisons on existing pedido

10. Agreed Product Roadmap (Phases 1–5)

Phase	Priority	Summary	Status
0 — Foundation	—	Auth, DB, upload, AI, deploy	✅ Done
1 — Atendimento	High	IA validator, programmatic checks, briefing	~85%
2 — Design	**Primary**	Checklist, versions, briefing link, artwork OCR	❌ Not built
3 — CQ	Secondary	Sketch validation, conditional approval	❌ Not built
4 — Metrics	Optional	Supplier score, KPIs, monthly reports	❌ Not built
5 — Integrations	Future	ERP, email, Teams	Out of scope 2026

11. FastAPI + MongoDB Migration Plan

11.1 Proposed FastAPI project structure

```
backend/
├── app/
│   ├── main.py                # FastAPI app, CORS, routers
│   ├── core/
│   │   ├── config.py          # pydantic-settings (env)
│   │   ├── security.py        # JWT, bcrypt
│   │   └── deps.py            # get_current_user, require_admin
│   ├── models/                # Pydantic request/response
│   ├── db/
│   │   ├── mongodb.py         # Motor client
│   │   └── collections.py     # Collection accessors
│   ├── routers/
│   │   ├── auth.py
│   │   ├── pedidos.py
│   │   ├── comparacoes.py     # ⭐ core
│   │   ├── documentos.py
│   │   ├── workflow.py
│   │   ├── prompts.py
│   │   ├── notificacoes.py
│   │   ├── aql.py
│   │   └── admin/
│   └── services/
│       ├── ocr.py             # port extrator-texto.ts
│       ├── validators.py      # port validador-*.ts
│       ├── kimi.py            # port comparacao executar
│       ├── pdf.py             # port pdf-generator
│       └── aql.py
├── requirements.txt
└── Dockerfile
```

11.2 Migration phases & timeline (with Cursor AI assistance)

Week	Deliverable
1	FastAPI skeleton + MongoDB + JWT auth + users collection
2	pedidos aggregate + upload/OCR + validators + Kimi executar
3	Remaining routers: workflow, pdf, notifications, aql, admin
4	React: replace tRPC with REST/OpenAPI client — page by page
5–6	Integration testing, permission fixes, deploy, MySQL data migration

Estimated total: 5–8 weeks (1 developer with Cursor) · 3–5 weeks (2 developers)

11.3 Python dependencies (suggested)

```
fastapi uvicorn[standard] motor pymongo pydantic pydantic-settings
python-jose[cryptography] passlib[bcrypt] httpx
pypdf pdfplumber pytesseract openpyxl python-multipart
playwright pytest
```

11.4 Port mapping — TypeScript files to Python

TypeScript source	Python target
api/local-auth.ts + local-auth-router.ts	routers/auth.py + core/security.py
api/comparacao-router.ts	routers/comparacoes.py + services/kimi.py
api/upload-router.ts + extrator-texto.ts	routers/documentos.py + services/ocr.py
api/lib/validador-*.ts	services/validators.py
api/workflow-router.ts	routers/workflow.py
api/pdf-router.ts + pdf-generator.ts	routers/pdf.py + services/pdf.py
db/schema.ts	Pydantic models + MongoDB document schemas

12. Environment, Deploy & Setup

12.1 Environment variables (current → FastAPI)

Variable	Purpose	Example
APP_ID	Moonshot API key	sk-...
APP_SECRET	JWT signing secret	random string
DATABASE_URL	MySQL (current)	mysql://...
MONGODB_URI	MongoDB (target)	mongodb+srv://...
KIMI_OPEN_URL	AI API base	https://api.moonshot.ai
MISTRAL_API_KEY	OCR (optional)	...

12.2 Current local dev (Node — reference until migration)

```
npm install
npm run db:migrate
npm run dev                    # http://localhost:3000
# Seed prompts:
curl "http://localhost:3000/api/trpc/setup.seed?input=..."
```

12.3 Production (was deployed)

- VPS: Ubuntu 22.04 · PM2 · MySQL Docker container `doccompare-mysql`
- GitHub Actions: push to main → SSH deploy → npm ci → build → pm2 restart
- Documented admin: admin@doccompare.com / admin123

13. Key Source Files Reference

Purpose	File path
AI comparison (CORE)	api/comparacao-router.ts
Upload + OCR trigger	api/upload-router.ts
OCR pipeline	api/extrator-texto.ts, api/lib/mistral-ocr.ts
Validators	api/lib/validador-programatico.ts, validador-cnpj/eán/ncm.ts
Workflow phases	api/workflow-router.ts
Auth + JWT	api/local-auth.ts, api/local-auth-router.ts
Permissions	api/middleware.ts
DB schema	db/schema.ts
Prompts seed	api/setup-router.ts
Upload UI	src/pages/NovaComparacao.tsx
Results UI	src/pages/Resultado.tsx
Client RFQ spec	RFQ_DOC_COMPARE.md
Phase 1 spec	ESPECIFICACAO_FASE1_FINAL.md

14. Recommended Work Order for Developer

Track A — Platform migration (FastAPI + MongoDB)

1. Scaffold FastAPI + Motor + JWT + `users` / `prompts` collections
2. Implement `pedidos` aggregate schema with embedded comparacoes
3. Port OCR service (api/extrator-texto.ts → services/ocr.py)
4. Port validators (api/lib/validador-*.ts → services/validators.py)
5. Port comparacao.executar (Kimi integration) — highest priority
6. Port workflow, pdf, notifications, aql, admin routers
7. Fix permission gaps (§9) during rewrite — do not port bugs
8. Replace tRPC in React with OpenAPI-generated client
9. GridFS or S3 for file storage
10. Deploy FastAPI alongside or replacing Node; migrate MySQL data if needed

Track B — Phase 1 polish (can run parallel week 1–2)

1. Wire Resumo Executivo PDF in Resultado UI
2. Enforce required document types for Phase 1
3. Phase ↔ prompt matching validation

Track C — Phase 2 Design (after migration stable)

1. Interactive per-field checklist (OK/NOK/Pending) stored in analises
2. Artwork version history UI
3. Phase 1 briefing panel in Design phase
4. Enhanced artwork OCR pipeline

End of document.
For questions about business rules, refer to `RFQ_DOC_COMPARE.md` and `ESPECIFICACAO_FASE1_FINAL.md` in the repository root.
Regenerate this PDF: `node scripts/generate-handoff-pdf.mjs`